# BÁO CÁO CƠ SỞ DỮ LIỆU

**Tên nhóm:** ROBOT

**Thành viên:**

Võ Hoài Phong (n24dccn151@student.ptithcm.edu.vn)

Trần Đăng Khôi (n24dccn136@student.ptithcm.edu.vn)

Nguyễn Phước An (n24dccn093@student.ptithcm.edu.vn)

**Tên đề tài:** Hệ Thống Quản Lý Kho Hàng & Chuỗi Cung Ứng

## NỘI DUNG BÁO CÁO

## 1. GIỚI THIỆU & PHẠM VI DỰ ÁN (THEO CHUẨN ISO/IEC/IEEE 29148)

### 1.0 Định Nghĩa Thuật Ngữ Cốt Lõi

**Warehouse (Nhà kho Vật lý & Nút giao Tồn kho):** Không chỉ là không gian lưu trữ vật lý, Warehouse là một điểm nút địa lý phân tán. Về mặt dữ liệu, nó quản lý sức chứa tối đa và neo giữ số liệu tồn kho thực tế độc lập cho từng loại vật tư để so sánh với ngưỡng an toàn.

**Supply Chain (Chuỗi Cung ứng & Luồng Dữ liệu):** Là mạng lưới liên kết thông tin nhằm luân chuyển hàng hóa. Về cấu trúc cơ sở dữ liệu, nó đại diện cho luồng tuyến tính: giao dịch với Nhà cung cấp (SUPPLIER), ghi nhận Đơn đặt hàng (ORDER), và theo dõi tiến trình Vận chuyển (SHIPMENT).

### 1.1 Mục Tiêu Hệ Thống

Hệ thống được thiết kế nhằm giải quyết bài toán theo dõi luồng hàng hóa phức tạp phân tán trên nhiều nhà kho địa lý khác nhau. Mục tiêu cốt lõi của cơ sở dữ liệu là quản lý chuỗi cung ứng từ khâu đặt hàng đến theo dõi vận chuyển, đồng thời xử lý cấu trúc sản phẩm đa cấp (BOM) và tự động hóa các ngưỡng cảnh báo, điều chuyển tồn kho.

### 1.2 Quy Tắc Nghiệp Vụ Cốt Lõi & Thách Thức Dữ Liệu

Hệ thống tập trung giải quyết các bài toán dữ liệu cốt lõi:

**Quản lý đa kho địa lý:** Số lượng mặt hàng phải được theo dõi độc lập tại từng vị trí nhà kho vật lý khác nhau thông qua bảng trung gian tồn kho (INVENTORY).

**Cấu trúc Đệ quy (BOM):** Khai báo các mặt hàng phức tạp được lắp ráp từ nhiều linh kiện con thông qua mối quan hệ tự trỏ (COMPOSED_OF). Hệ thống bắt buộc phải ghi nhận chính xác định mức định lượng (ComponentQty) của từng linh kiện để cấu thành nên một sản phẩm cha.

**Kích hoạt cảnh báo:** Tại mỗi kho, từng mặt hàng có ngưỡng báo động (ReorderPoint) riêng biệt để hệ thống kích hoạt cảnh báo tự động khi lượng tồn rớt xuống dưới mức an toàn.

**Cơ chế Điều chuyển nội bộ:** Bảng trung gian INVENTORY đóng vai trò là cơ sở đối chiếu tồn kho toàn mạng lưới. Về mặt chính sách, hệ thống bắt buộc phải ưu tiên luân chuyển hàng hóa nội bộ (INTERNAL_TRANSFER) giữa các nút địa lý trước khi phát sinh giao dịch đặt mua ngoài (ORDER).

**Ràng buộc Quản lý & Đối tác:** Mỗi nhà kho vật lý chỉ được phép quản lý trực tiếp bởi duy nhất một nhân viên kho (WAREHOUSE_STAFF) nhằm đảm bảo quy trách nhiệm (1-1). Tuy nhiên, với đối tác bên ngoài, một Nhà cung cấp có thể có nhiều người liên hệ đại diện (SUPPLIERCONTACT) khác nhau (1-N).

### 1.3 Yêu Cầu Hệ Thống (Chức Năng & Phi Chức Năng)

**Yêu cầu chức năng (FR):**

**FR01 (Quản lý tồn kho):** Hệ thống phải cho phép Nhân viên kho cập nhật số lượng tồn kho và thiết lập ngưỡng an toàn cho từng mặt hàng.

**FR02 (Xử lý đơn hàng):** Hệ thống phải tự động theo dõi trạng thái đơn hàng và cập nhật lịch trình giao hàng. **FR03 (Phân rã BOM):** Hệ thống phải có khả năng truy vấn cấu trúc đệ quy đa cấp để phân rã một sản phẩm cha thành các linh kiện con.

**FR04 (Cân bằng tồn kho):** Khi một kho cạn hàng (StockQuantity < ReorderPoint), hệ thống tự động quét kiểm tra các kho khác. Nếu phát hiện kho nào đang dư hàng, hệ thống ưu tiên tạo "Lệnh điều chuyển nội bộ" để luân chuyển hàng hóa. Ngược lại, nếu toàn bộ mạng lưới kho đều không có hàng dư thừa, hệ thống mới tự động tạo Đơn đặt hàng (ORDER) gửi cho Nhà cung cấp ngoài.

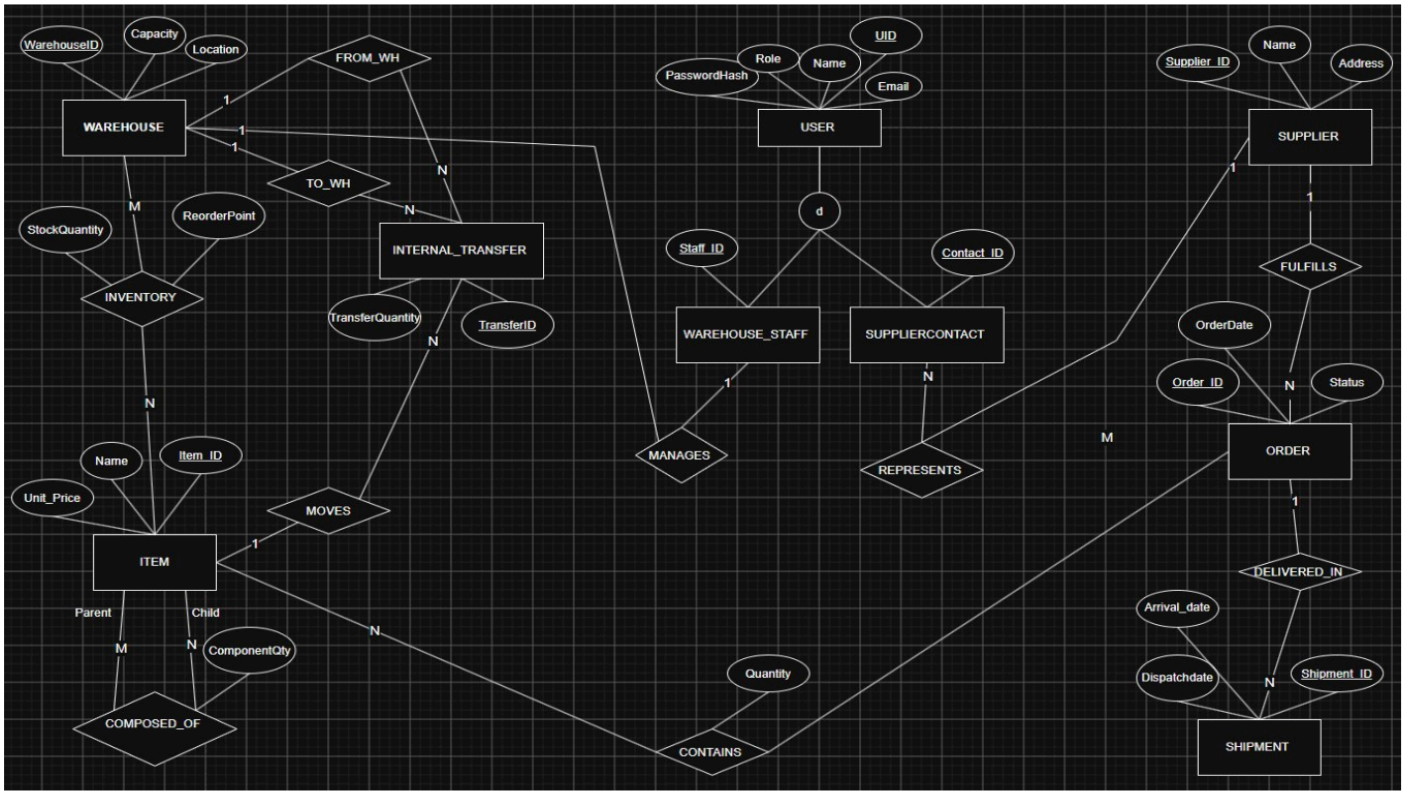
**FR05 (Quản lý Người dùng & Phân quyền):** Hệ thống bắt buộc phải xác thực tài khoản và phân cấp quyền hành (Role) truy cập dựa trên phân rã thực thể. Cụ thể, Nhân viên kho (WAREHOUSE_STAFF) chỉ được phép cập nhật tồn kho tại nhà kho mình quản lý trực tiếp (chuẩn hóa tỷ lệ 1-1), trong khi Đại diện đối tác (SUPPLIERCONTACT) chỉ được phép theo dõi tiến độ các Đơn đặt hàng liên quan đến nhà cung cấp của họ.

**Yêu cầu phi chức năng (NFR):**

**NFR01 (Bảo mật):** Mật khẩu người dùng bắt buộc phải được mã hóa (PasswordHash) tại cơ sở dữ liệu.
**NFR02 (Hiệu năng):** Các truy vấn kiểm tra tình trạng tồn kho đa kho phải phản hồi dưới mức 200ms. **NFR03 (Toàn vẹn dữ liệu):** Đảm bảo tính toàn vẹn dữ liệu tham chiếu thông qua các Khóa ngoại nghiêm ngặt.

# 2. THIẾT KẾ CƠ SỞ DỮ LIỆU (THEO CHUẨN ISO/IEC 19505 / IE)

## 2.1 Mô Hình Khái Niệm (Sơ đồ EER)




## 2.2 Ánh Xạ Lược Đồ Lô-gíc

**users** (uid, name, email, password_hash, role)

**warehouse_staff** (staff_id, uid [FK])

**supplier_contact** (contact_id, uid [FK], supplier_id [FK])

**warehouse** (warehouse_id, location, capacity, staff_id [FK])

**item** (item_id, name, unit_price)

**inventory** (warehouse_id [FK], item_id [FK], stock_quantity, reorder_point)

**composed_of** (parent_item_id [FK], child_item_id [FK], component_qty)

**supplier** (supplier_id, name, address)

**order** (order_id, order_date, status, supplier_id [FK])

**shipment** (shipment_id, dispatch_date, arrival_date, order_id [FK])

**contains** (order_id [FK], item_id [FK], quantity)

**internal_transfer** (transfer_id, from_warehouse_id [FK], to_warehouse_id [FK], item_id [FK], transfer_quantity)

### 2.3 Kiểm Chứng Chuẩn Hóa (Chứng minh hình thức từ 1NF đến 3NF/BCNF)

| Dạng Chuẩn | Đánh Giá Tình Trạng Lược Đồ |
|---|---|
| **1NF** | Tất cả các thuộc tính trong tất cả các bảng đều chứa giá trị nguyên tố (Atomic values). Các danh sách sản phẩm trong đơn hàng, hay lịch sử điều chuyển giữa các kho bãi đều đã được tách riêng ra các bảng trung gian. Đạt 1NF. |
| **2NF** | Mọi thuộc tính không khóa đều phụ thuộc hàm đầy đủ vào toàn bộ khóa chính. Trong các bảng có khóa kết hợp như inventory, thuộc tính StockQuantity phụ thuộc vào cả cặp khóa (mã kho và mã hàng) chứ không phụ thuộc vào riêng từng thành phần. Đạt 2NF. |
| **3NF** | Không tồn tại phụ thuộc bắc cầu (Transitive Dependency) giữa các thuộc tính không khóa. Ví dụ: Địa chỉ nhà cung cấp không lưu ở bảng order mà được bóc tách triệt để sang bảng supplier. Đạt 3NF. |
| **BCNF** | Với mọi phụ thuộc hàm có dạng X → A, X đều là một Siêu khóa (Super Key) hoặc Khóa chính (Primary Key). Lược đồ hoàn toàn tối ưu cho nghiệp vụ chuỗi cung ứng và chống trùng lặp dữ liệu. |

## 3. TỪ ĐIỂN DỮ LIỆU (THEO CHUẨN ISO/IEC 11179)

(Bảng từ điển dữ liệu định nghĩa các thuộc tính cốt lõi của hệ thống Quản lý Tồn kho & Điều chuyển)

| Tên Thuộc Tính | Kiểu Dữ Liệu | Loại Khóa | Cho Phép Null? | Giá Trị Mặc Định | Quy Tắc Nghiệp Vụ / Ràng Buộc |
|---|---|---|---|---|---|
| warehouse_id | INT | PK | Không | Không có | Cấp phát tự động, bắt buộc đối với mọi kho. |
| capacity | INT | Không có | Không | 100 | Sức chứa tối đa của kho, bắt buộc ≥ 0. |
| item_id | INT | PK/FK | Không | Không có | Tham chiếu bảng item. |
| stock_quantity | INT | Không có | Không | 0 | Số lượng tồn kho thực tế, bắt buộc ≥ 0. |
| reorder_point | INT | Không có | Không | 10 | Ngưỡng báo động, bắt buộc ≥ 0. Kích hoạt truy vấn FR04 khi cạn hàng. |
| transfer_id | INT | PK | Không | Không có | Mã lệnh điều chuyển, tự động tăng (Auto Increment). |
| from_warehouse_id | INT | FK | Không | Không có | Mã kho xuất hàng, tham chiếu warehouse(warehouse_id). |
| to_warehouse_id | INT | FK | Không | Không có | Mã kho nhận hàng, tham chiếu warehouse(warehouse_id). |
| transfer_quantity | INT | Không có | Không | 1 | Số lượng vật tư luân chuyển nội bộ, bắt buộc ≥ 1. |

# 4. CÀI ĐẶT CƠ SỞ DỮ LIỆU (SQL STYLE GUIDE COMPLIANT)

## 4.1 DDL Script (Tạo Cấu Trúc Bảng, Khóa & Ràng Buộc)

Script SQL tuân thủ chuẩn đặt tên snake_case, định kiểu dữ liệu tối ưu và ràng buộc toàn vẹn nghiêm ngặt.

```sql
-- KÍCH HOẠT CƠ SỞ DỮ LIỆU & TẠO BẢNG CỐT LÕI
CREATE TABLE users (
    uid INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('STAFF', 'CONTACT', 'ADMIN') NOT NULL
);

CREATE TABLE warehouse_staff (
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
    uid INT UNIQUE NOT NULL,
    FOREIGN KEY (uid) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE supplier (
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    address VARCHAR(255) NOT NULL
);

CREATE TABLE supplier_contact (
    contact_id INT AUTO_INCREMENT PRIMARY KEY,
    uid INT UNIQUE NOT NULL,
    supplier_id INT NOT NULL,
    FOREIGN KEY (uid) REFERENCES users(uid) ON DELETE CASCADE,
    FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id)
);

CREATE TABLE warehouse (
    warehouse_id INT AUTO_INCREMENT PRIMARY KEY,
    location VARCHAR(255) NOT NULL,
    capacity INT NOT NULL DEFAULT 100 CHECK (capacity >= 0),
    staff_id INT UNIQUE NOT NULL,
    FOREIGN KEY (staff_id) REFERENCES warehouse_staff(staff_id)
);

CREATE TABLE item (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    unit_price DECIMAL(12, 2) NOT NULL CHECK (unit_price >= 0)
);

CREATE TABLE inventory (
    warehouse_id INT NOT NULL,
    item_id INT NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    reorder_point INT NOT NULL DEFAULT 10 CHECK (reorder_point >= 0),
    PRIMARY KEY (warehouse_id, item_id),
    FOREIGN KEY (warehouse_id) REFERENCES warehouse(warehouse_id),
    FOREIGN KEY (item_id) REFERENCES item(item_id)
);
-- CÁC BẢNG LIÊN KẾT BOM, ĐƠN HÀNG, VẬN CHUYỂN VÀ ĐIỀU CHUYỂN NỘI BỘ
CREATE TABLE composed_of (
    parent_item_id INT NOT NULL,
    child_item_id INT NOT NULL,
    component_qty INT NOT NULL DEFAULT 1 CHECK (component_qty > 0),
    PRIMARY KEY (parent_item_id, child_item_id),
    FOREIGN KEY (parent_item_id) REFERENCES item(item_id),
    FOREIGN KEY (child_item_id) REFERENCES item(item_id)
);

CREATE TABLE `order` (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status ENUM('PENDING', 'SHIPPED', 'DELIVERED', 'CANCELLED') DEFAULT 'PENDING',
    supplier_id INT NOT NULL,
```

```
    FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id)
);

CREATE TABLE contains (
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (order_id, item_id),
    FOREIGN KEY (order_id) REFERENCES `order`(order_id),
    FOREIGN KEY (item_id) REFERENCES item(item_id)
);

CREATE TABLE shipment (
    shipment_id INT AUTO_INCREMENT PRIMARY KEY,
    dispatch_date DATETIME,
    arrival_date DATETIME,
    order_id INT UNIQUE NOT NULL,
    FOREIGN KEY (order_id) REFERENCES `order`(order_id)
);

CREATE TABLE internal_transfer (
    transfer_id INT AUTO_INCREMENT PRIMARY KEY,
    from_warehouse_id INT NOT NULL,
    to_warehouse_id INT NOT NULL,
    item_id INT NOT NULL,
    transfer_quantity INT NOT NULL DEFAULT 1 CHECK (transfer_quantity > 0),
    transfer_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (from_warehouse_id) REFERENCES warehouse(warehouse_id),
    FOREIGN KEY (to_warehouse_id) REFERENCES warehouse(warehouse_id),
    FOREIGN KEY (item_id) REFERENCES item(item_id)
);
```

### 4.2 DML Script (Chèn Dữ Liệu Mẫu Kiểm Thử)

```
-- CHÈN DỮ LIỆU MẪU
INSERT INTO users (name, email, password_hash, role) VALUES
('Nguyen Van A', 'a.nguyen@warehouse.com', 'hash_pass_1', 'STAFF'),
('Tran Thi B', 'b.tran@warehouse.com', 'hash_pass_2', 'STAFF'),
('Le Van C', 'c.le@supplier.com', 'hash_pass_3', 'CONTACT');

INSERT INTO warehouse_staff (uid) VALUES (1), (2);
INSERT INTO supplier (name, address) VALUES ('Cung Cap Linh Kien HCM', '123 Le Van Viet, Q9,
HC M');
INSERT INTO supplier_contact (uid, supplier_id) VALUES (3, 1);

INSERT INTO warehouse (location, capacity, staff_id) VALUES
('Kho Thu Duc - HCM', 500, 1),
('Kho Binh Duong', 1000, 2);

INSERT INTO item (name, unit_price) VALUES
```

### 4.3 Các Truy Truy Vấn Nâng Cao & Kiểm Thử Hiệu Năng

**Truy vấn 1 (Phân rã BOM đệ quy - Đáp ứng yêu cầu FR03):** Truy vấn danh sách toàn bộ các linh kiện cấu thành nên sản phẩm "Robot Arm Assembly" kèm định lượng.

```
WITH RECURSIVE bom_tree AS (
    SELECT
        parent_item_id, child_item_id, component_qty, 1 AS level
    FROM composed_of
    WHERE parent_item_id = 1
    UNION ALL
    SELECT
        c.parent_item_id, c.child_item_id, c.component_qty, b.level + 1
    FROM composed_of c
    INNER JOIN bom_tree b ON c.parent_item_id = b.child_item_id
)
SELECT
    p.name AS parent_product,
    i.name AS component_name,
    b.component_qty,
    b.level
FROM bom_tree b
JOIN item p ON b.parent_item_id = p.item_id
```

```
JOIN item i ON b.child_item_id = i.item_id;
```

**Truy vấn 2 (Quét cân bằng tồn kho tự động - FR04):** Phát hiện các mặt hàng dưới ngưỡng an toàn tại Kho A và tìm kiếm Kho B đang có lượng dư tồn kho lớn hơn 0 để sẵn sàng tạo lệnh điều chuyển.

```
SELECT
    inv_a.warehouse_id AS low_stock_warehouse,
    inv_a.item_id,
    item.name AS item_name,
    inv_a.stock_quantity AS current_stock,
    inv_a.reorder_point,
    inv_b.warehouse_id AS surplus_warehouse,
    inv_b.stock_quantity AS available_surplus
FROM inventory inv_a
JOIN item ON inv_a.item_id = item.item_id
JOIN inventory inv_b ON inv_a.item_id = inv_b.item_id
    AND inv_a.warehouse_id <> inv_b.warehouse_id
WHERE inv_a.stock_quantity < inv_a.reorder_point
  AND inv_b.stock_quantity > inv_b.reorder_point;
```

### 4.4 Triggers & Views (Tự Động Hóa Nghiệp Vụ)

**Trigger 1: Tự động cập nhật tồn kho khi hoàn tất điều chuyển nội bộ (INTERNAL_TRANSFER)**

```
DELIMITER //
CREATE TRIGGER trg_after_internal_transfer
AFTER INSERT ON internal_transfer
FOR EACH ROW
BEGIN
    -- Trừ tồn kho tại kho xuất
    UPDATE inventory
    SET stock_quantity = stock_quantity - NEW.transfer_quantity
    WHERE warehouse_id = NEW.from_warehouse_id AND item_id = NEW.item_id;

    -- Cộng tồn kho tại kho nhận
    UPDATE inventory
    SET stock_quantity = stock_quantity + NEW.transfer_quantity
    WHERE warehouse_id = NEW.to_warehouse_id AND item_id = NEW.item_id;
END;
//
DELIMITER ;
```

**View 1: Báo cáo tổng hợp tình trạng tồn kho đa kho và mức cảnh báo (Real-time Dashboard)**

```
CREATE VIEW vw_inventory_alert_dashboard AS
SELECT
    w.warehouse_id,
    w.location,
    i.item_id,
    i.name AS item_name,
    inv.stock_quantity,
    inv.reorder_point,
    CASE
        WHEN inv.stock_quantity = 0 THEN 'CRITICAL_OUT_OF_STOCK'
        WHEN inv.stock_quantity < inv.reorder_point THEN 'WARNING_REORDER'
        ELSE 'NORMAL'
    END AS stock_status
FROM inventory inv
JOIN warehouse w ON inv.warehouse_id = w.warehouse_id
JOIN item i ON inv.item_id = i.item_id;
```

## 5. KIỂM THỬ & BẢO MẬT CƠ SỞ DỮ LIỆU

### 5.1 Kiểm Thử Ràng Buộc Dữ Liệu (Constraint Test Cases)

Đảm bảo CSDL chủ động ngăn chặn các dữ liệu vi phạm nghiệp vụ:

> **Test Case 1 (Âm số lượng tồn kho):** Lệnh `INSERT INTO inventory (warehouse_id, item_id, stock_quantity) VALUES (1, 2, -5);` bị hệ thống chặn bởi ràng buộc `CHECK (stock_quantity >= 0)`.

**Test Case 2 (Trùng lặp Nhân viên quản lý kho):** Lệnh gán 1 `staff_id` cho 2 kho khác nhau bị chặn bởi ràng buộc `UNIQUE (staff_id)` ở bảng `warehouse`.

## 5.2 Phân Quyền Truy Cấp Dựa Trên Vai Trò (RBAC Definition)

Thực thi nguyên tắc tối thiểu đặc quyền (Principle of Least Privilege) theo đúng yêu cầu FR05:

```
-- 1. TẠO CÁC VAI TRÒ (ROLES)
CREATE ROLE role_warehouse_staff;
CREATE ROLE role_supplier_contact;

-- 2. CẤP QUYỀN CHO NHÂN VIÊN KHO (Chỉ thao tác Tồn kho và Điều
chuyển) GRANT SELECT, UPDATE ON inventory TO role_warehouse_staff;
GRANT SELECT, INSERT ON internal_transfer TO role_warehouse_staff;
GRANT SELECT ON item TO role_warehouse_staff;
GRANT SELECT ON vw_inventory_alert_dashboard TO role_warehouse_staff;

-- 3. CẤP QUYỀN CHO ĐẠI DIỆN NHÀ CUNG CẤP (Chỉ theo dõi Đơn hàng & Vận
chuyển) GRANT SELECT ON `order` TO role_supplier_contact;
GRANT SELECT ON contains TO role_supplier_contact;
GRANT SELECT, UPDATE ON shipment TO role_supplier_contact;

-- 4. GÁN VAI TRÒ CHO TÀI KHOẢN NGƯỜI DÙNG CỤ THỂ
CREATE USER 'staff_user1'@'localhost' IDENTIFIED BY 'SecureStaffPass123!';
GRANT role_warehouse_staff TO 'staff_user1'@'localhost';

CREATE USER 'supplier_user1'@'localhost' IDENTIFIED BY 'SecureSupplierPass123!';
GRANT role_supplier_contact TO 'supplier_user1'@'localhost';
```